

Car Rental System

Project Report

Team members:

Naguib Shokry – 42410156

Marina Nabil – 42410233

Barthena Bassim – 42410091

Sara George – 42410231

Dina Ashraf – 42410313

Philopater Rashad – 42410020

Mariam Mohamed – 42410303

The Car Rental Management System is a console-based application designed to manage the operations of a car rental company efficiently. The system provides functionalities for storing car information, managing customers, renting and returning vehicles, and saving/loading data using files. The project focuses on applying core data structures concepts such as Linked Lists, Stacks, Queues, and Binary Search Trees (BST) in a real-life scenario.

The main goal of this project is to simulate how a real car rental system works, while demonstrating a strong understanding of data structures, object-oriented programming, templates, and file handling in C++.

- **Problem Definition:**

Traditional manual car rental systems face several problems such as:

- Difficulty in organizing car records.
- Slow searching and sorting operations.
- Poor tracking of rented and available cars.
- Inefficient customer handling.
- Risk of losing data without persistent storage.

This project aims to solve these issues by providing:

- Fast searching using Binary Search Trees.
- Organized car storage using Linked Lists.
- Proper handling of rented cars using Stacks.
- Queue-based customer waiting management.
- Permanent storage using text files.

- **System objectives:**

The main objectives of the system are:

1. Store and manage car information (ID, name, price, availability).
2. Allow searching for cars efficiently by ID.
3. Sort cars based on different criteria (ID, price, name).
4. Manage customer waiting lists.
5. Handle car renting and returning processes.
6. Save and load data using files to maintain persistence.
7. Apply multiple data structures correctly in one integrated system.

- **System Design Overview:**

The system is designed using **Object-Oriented Programming (OOP)** principles. Each part of the system is separated into logical components, which improves readability, maintainability, and scalability.

The main components of the system are:

- Car entity
- Linked List for car storage
- Binary Search Tree for fast searching and sorting by ID
- Stack for rented cars
- Queue for customers
- File handling module
- Menu-driven user interface

- **Car Entity Design:**

The Car entity represents the core data model of the system. Each car has:

- A unique ID
- A name
- A rental price
- Availability status (available or rented)

This abstraction allows the system to manage cars consistently across all data structures. Encapsulation is applied by keeping attributes private and using public setters and getters to access or modify data safely.

- **Linked List for Car Storage:**

A singly linked list is used to store all cars in the system. This structure was chosen because:

- It allows dynamic memory allocation.
- Cars can be added or removed easily.
- It supports traversal for display and sorting operations.

The linked list is responsible for:

- Adding new cars.
- Deleting cars by name.
- Displaying all cars.
- Sorting cars by price.
- Sorting cars by name.
- Updating car availability during rent/return operations.

Bubble Sort logic is applied for sorting, which is acceptable due to the limited size of the dataset.

- **Binary Search Tree (BST):**

The Binary Search Tree is used to organize cars based on their ID. This structure provides:

- Fast searching ($O(\log n)$ in average case).
- Automatic sorting by ID using inorder traversal.

The BST supports:

- Inserting cars while preventing duplicate IDs.
- Searching for cars by ID.
- Displaying cars sorted by ID.
- Deleting cars by ID.

Using a BST improves performance significantly compared to linear searching.

- **Stack for Rented Cars**

A Stack data structure is used to store recently rented cars. This follows the **Last In, First Out (LIFO)** principle, which is suitable for:

- Tracking the most recently rented cars.
- Displaying recent rental history.

The stack supports:

- Pushing rented cars.
- Popping cars during return operations.
- Peeking at the top rented car.
- Displaying all rented cars.

This design helps simulate real-world rental tracking.

- **Queue for Customers**

Customers are handled using a Queue data structure, following the **First In, First Out (FIFO)** principle. This ensures fairness when multiple customers are waiting.

The queue manages:

- Enqueuing new customers.
- Dequeuing customers when a car is rented.
- Displaying waiting customers.
- Clearing the queue when needed.

This approach models real customer service systems accurately.

- **File Handling and Data Persistence:**

To ensure data is not lost when the program terminates, file handling is implemented using text files.

The system stores:

- Car data in a file.
- Customer data in a separate file.

Features include:

- Saving individual records.
- Overwriting entire files when updates occur.
- Loading data at program startup.

This guarantees persistence and continuity across program runs.

- **Rental and Return Logic:**

Renting a Car

The rent process includes:

1. Checking if there are waiting customers.
2. Searching for the car by ID.
3. Ensuring the car is available.
4. Updating availability status.
5. Adding the car to the rented stack.
6. Removing the customer from the queue.
7. Updating files.

Returning a Car

The return process includes:

1. Searching rented cars using a temporary stack.
2. Restoring the original stack order.
3. Updating car availability.
4. Saving updated data to files.

This logic ensures data consistency across all structures.

- **User Interface (Menu System):**

The system uses a menu-driven interface that allows users to:

- View all cars.
- Search for cars.
- Sort cars.
- Add cars and customers.
- Rent and return cars.
- View rented cars.
- View waiting customers.

This interface makes the system user-friendly and easy to navigate.

- **Error Handling:**

Basic error handling is applied to:

- Prevent operations on empty stacks or queues.
- Handle invalid search results.
- Avoid duplicate IDs.
- Detect file access issues.

This improves system reliability and robustness.

Conclusion

The Car Rental Management System successfully demonstrates the integration of multiple data structures into a single real-world application. It provides efficient searching, sorting, and data management while maintaining clean design and logical separation of concerns.

The project reflects strong understanding of:

- Object-Oriented Programming
- Data Structures
- Templates
- File Handling
- System Design

Overall, this system serves as a solid academic project that simulates a practical application while reinforcing fundamental computer science concepts.

